Laboratorio 12: Teoría de la Computación

Respuestas teóricas del Problema 1
Noviembre 2026

La implementación está escrita en JavaScript (Node.js), en el archivo `problema1/problema1.js`.

En este documento se usa la notación usual del cálculo lambda: `λx.M` es una función con parámetro `x` y cuerpo `M`; `M N` es aplicar `M` a `N`; la aplicación asocia a la izquierda (`M N P = (M N) P`) y el cuerpo de una lambda se extiende lo más posible a la derecha. La **reducción-β** es la regla de cómputo:

```
(λx.M) N  →β  M[x := N]
```

Es decir, se reemplaza cada aparición libre de `x` en `M` por `N`.

A. Reducción-β de la operación lógica NOT

A.1 Booleanos de Church

El cálculo lambda puro no tiene valores primitivos: los booleanos se codifican como funciones que *eligen* entre dos alternativas. `TRUE` devuelve la primera y `FALSE` la segunda, por lo que un booleano se comporta como un *if-then-else*: `B M N` da `M` si `B` es verdadero y `N` si es falso.

```
TRUE  ≡ λx.λy.x
FALSE ≡ λx.λy.y
NOT   ≡ λb.b FALSE TRUE
```

`NOT` se lee como "si `b` entonces `FALSE`, si no `TRUE`".

A.2 NOT TRUE

#	Término	Explicación
0	`NOT TRUE`	Expresión inicial.
δ	`(λb.b FALSE TRUE) (λx.λy.x)`	Se expanden las definiciones. El término completo es un redex: una abstracción aplicada a un argumento.
1	`→β TRUE FALSE TRUE`	`[b := TRUE]`: en el cuerpo `b FALSE TRUE` se reemplaza `b` por `TRUE`.
	`= (λx.λy.x) FALSE TRUE`	Se expande `TRUE`. Como la aplicación asocia a la izquierda, el redex es `(λx.λy.x) FALSE`.
2	`→β (λy.FALSE) TRUE`	`[x := FALSE]`: el cuerpo `λy.x` se convierte en `λy.FALSE`.
3	`→β FALSE`	`[y := TRUE]`: `y` no aparece libre en `FALSE`, así que el argumento se descarta.

Resultado: `NOT TRUE ↠β FALSE = λx.λy.y`, una forma normal (ya no contiene redexes) obtenida en 3 pasos β.

A.3 NOT FALSE

#	Término	Explicación
0	`NOT FALSE = (λb.b FALSE TRUE) (λx.λy.y)`	Expresión inicial con definiciones expandidas.
1	`→β FALSE FALSE TRUE`	`[b := FALSE].`
	`= (λx.λy.y) FALSE TRUE`	Se expande el primer `FALSE`.
2	`→β (λy.y) TRUE`	`[x := FALSE]`: `x` no aparece en `λy.y`, así que el primer argumento se descarta.
3	`→β TRUE`	`[y := TRUE]`: la identidad devuelve su argumento.

Resultado: `NOT FALSE ↠β TRUE = λx.λy.x`. Con esto se comprueba la tabla de verdad completa de NOT.

A.4 Definición alternativa

Otra codificación común de NOT intercambia directamente las dos alternativas del booleano: `NOT' ≡ λp.λa.λb.p b a`. Su reducción muestra que los pasos β también pueden ocurrir dentro de una abstracción:

```
NOT' TRUE = (λp.λa.λb.p b a) (λx.λy.x)
   →β λa.λb.(λx.λy.x) b a          [p := TRUE]
   →β λa.λb.(λy.b) a               [x := b]
   →β λa.λb.b                      [y := a]   ≡α  λx.λy.y = FALSE
```

El resultado `λa.λb.b` es igual a `FALSE` salvo el nombre de las variables ligadas (α-equivalencia). Por eso ambas definiciones de NOT son equivalentes.

El programa `problema1/problema1.js` incluye un pequeño reductor de cálculo lambda (orden normal, con sustitución que evita la captura de variables). Ese reductor calcula estas reducciones y las imprime paso a paso. También reduce `NOT (NOT TRUE) ↠β TRUE` en 6 pasos. Las pruebas automatizadas (`node --test`) confirman los resultados.

B. Recursión y ciclos

B.1 Recursión

En la programación funcional, la recursión es el mecanismo básico de repetición: una función se define en términos de sí misma, con un **caso base** que detiene el proceso y un **caso recursivo** que se acerca al caso base. En JavaScript esto se escribe directamente porque la lambda tiene un nombre al que puede referirse:

```
const factorial = n => n === 0 ? 1 : n * factorial(n - 1);
factorial(5);   // 120
```

El problema en cálculo lambda. Las funciones lambda son *anónimas*, así que una función no puede referirse a sí misma por nombre. La solución es un **combinador de punto fijo**. El más conocido es el combinador Y de Curry:

```
Y ≡ λf.(λx.f (x x)) (λx.f (x x))
```

Su propiedad fundamental se obtiene por reducción-β:

```
Y g = (λf.(λx.f (x x)) (λx.f (x x))) g
   →β (λx.g (x x)) (λx.g (x x))              [f := g]
   →β g ((λx.g (x x)) (λx.g (x x)))          [x := λx.g (x x)]
    =β g (Y g)
```

Es decir, `Y g` es un punto fijo de `g`: `Y g =β g (Y g) =β g (g (Y g)) =β ...`. Si `g` recibe como primer parámetro "la función que se llama a sí misma", `Y` se encarga de pasársela. El factorial queda así:

```
FACT ≡ Y (λf.λn. ISZERO n 1 (MULT n (f (PRED n))))

FACT 2 =β (λn. ISZERO n 1 (MULT n (FACT (PRED n)))) 2
       =β MULT 2 (FACT 1)
       =β MULT 2 (MULT 1 (FACT 0))
       =β MULT 2 (MULT 1 1)  =β  2
```

Aquí `ISZERO` devuelve un booleano de Church, que actúa como *if*. Esa elección es justamente la que detiene la recursión en el caso base.

Recursión en un lenguaje estricto. JavaScript evalúa los argumentos antes de llamar a una función (evaluación estricta). Por eso `Y` escrito tal cual intenta evaluar `x(x)` infinitamente y produce `RangeError: Maximum call stack size exceeded`. La versión para lenguajes estrictos es el **combinador Z**, que envuelve la autoaplicación en una lambda (η-expansión) para retrasarla hasta que se necesita:

```
const Z = f => (x => f(v => x(x)(v)))(x => f(v => x(x)(v)));
const factorialZ = Z(self => n => n === 0 ? 1 : n * self(n - 1));
factorialZ(5);   // 120
```

B.2 Ciclos

En el paradigma funcional no hay variables que cambien de valor. Por eso un ciclo `for`/`while` "clásico", que depende de un contador mutable, no tiene un equivalente directo. Las iteraciones se expresan de tres formas:

1. **Recursión de cola con acumulador.** El estado del ciclo (contador y resultado parcial) viaja en los parámetros. La llamada recursiva es lo último que se ejecuta, así que no queda trabajo pendiente y el compilador puede convertirla en un salto, igual que un ciclo.

   ```
   // imperativo: el contador i y el total cambian en cada vuelta
   let total = 0;
   for (let i = 1; i <= n; i++) total += i;

   // funcional (recursión de cola): el estado va en los parámetros
   const suma = (n, acc = 0) =>
     n === 0 ? acc : suma(n - 1, acc + n);
   ```

2. **Funciones de orden superior.** Los patrones de ciclo más comunes ya existen como funciones que reciben una lambda: `map` (transformar cada elemento), `filter`

(seleccionar) y `reduce` (acumular). El índice y la mutación quedan ocultos:

```
const rango = (a, b) =>
  Array.from({ length: b - a + 1 }, (_, i) => a + i);
rango(1, 10).reduce((acc, x) => acc + x, 0);     // 55
```

3. **Numerales de Church (cálculo lambda).** El número *n* se representa como "aplicar `f` *n* veces": `n ≡ λf.λx.f (f (… (f x)))`. Así, un número *es* un ciclo acotado. Por ejemplo, `3 f x = f (f (f x))`, y en JavaScript `tres(s => s + "*")("")` da `"***"`.

Forma	**Estado**	**Ventaja**	**Limitación**
Ciclo `for` imperativo	Variables mutables	Eficiente y directo	Efectos secundarios; más difícil de razonar
Recursión (de cola)	Parámetros	Sin mutación; se demuestra por inducción	En lenguajes sin optimización de llamadas de cola desborda la pila
`map`/`filter`/`reduce`	Oculto en la función	Declarativo y fácil de componer	Puede crear listas intermedias
Numerales de Church	Ninguno	Base teórica: los ciclos son funciones	Solo es práctico como modelo teórico

Diferencia clave: la recursión general (con `Y`) puede no terminar. Por ejemplo, `Y g` no tiene forma normal. Un ciclo con numeral de Church o con `reduce` sobre una lista finita siempre termina, porque el número de vueltas está fijado de antemano.

C. ¿Cuándo es prudente usar programación funcional y cuándo no?

Cuándo sí conviene

- **Transformación de datos.** Procesos que reciben datos y producen otros datos (reportes, ETL, APIs que transforman JSON) se expresan bien como una cadena de `map`/`filter`/`reduce`.
- **Concurrencia y paralelismo.** Si los datos son inmutables, varios hilos no pueden pisarse entre sí, así que desaparecen las condiciones de carrera. Por eso Erlang/Elixir se usan en telecomunicaciones, y modelos como MapReduce/Spark se basan en funciones puras.
- **Código que debe ser confiable y fácil de probar.** Una función pura siempre devuelve lo mismo para la misma entrada, así que se prueba sin preparar ningún estado. También se razona con sustitución y con inducción, igual que la reducción-β.

- **Compiladores, intérpretes y analizadores.** Recorren estructuras recursivas (árboles), que se procesan naturalmente con funciones recursivas. Un ejemplo es el reductor de cálculo lambda de este laboratorio.

Ejemplo: calcular el total de ventas por categoría a partir de una lista de transacciones. No hay variables compartidas, cada paso se puede probar por separado y el mismo código funciona si los datos se reparten entre varios procesos.

```
const totalPorCategoria = (ventas) => ventas
  .filter(v => v.estado === 'pagada')
  .reduce((acc, v) => ({
    ...acc,
    [v.categoria]: (acc[v.categoria] ?? 0) + v.monto,
  }), {});
```

Cuándo no conviene

- **Sistemas dominados por estado mutable y rendimiento en tiempo real**, como motores de videojuegos, simulaciones físicas o sistemas embebidos con poca memoria. Copiar estructuras inmutables en cada cambio aumenta el uso de memoria y el trabajo del recolector de basura.
- **Algoritmos que son naturalmente *in-place***, como quicksort sobre un arreglo, union-find o la programación dinámica sobre tablas grandes. La versión imperativa es más simple y más rápida.
- **Recursión profunda en lenguajes sin optimización de llamadas de cola**, como JavaScript/Node (V8) o Python. Una "iteración" recursiva de cien mil vueltas desborda la pila.
- **Programas centrados en entrada/salida** (scripts de sistema, drivers) o equipos sin experiencia en el paradigma, donde el código funcional puede quedar menos claro que el equivalente imperativo.

Ejemplo: sumar los números del 1 al 100 000 con recursión de cola en Node.js. La versión funcional falla, aunque la idea es correcta, mientras que el ciclo imperativo termina sin problema. Esto se comprobó ejecutando el código de `problema1.js`:

```
sumaRecursiva(100000)    // RangeError
sumaImperativa(100000)   // 5000050000
```

Otro ejemplo es actualizar la posición de 10 000 partículas 60 veces por segundo. Crear un arreglo nuevo en cada fotograma desperdicia memoria. Modificar el arreglo existente (estilo imperativo) es la opción adecuada.

Conclusión: la programación funcional es prudente cuando importan la corrección, la composición de transformaciones y la concurrencia segura. Es menos adecuada cuando el problema es, por naturaleza, de estado mutable, de rendimiento extremo o de recursión muy profunda en un lenguaje que no la optimiza. En la práctica, muchos lenguajes (JavaScript, Java, Rust, Kotlin) permiten combinar ambos estilos y elegir el más claro en cada caso.